

L'EPOPEE DES SILENCES

Spécifications Fonctionnelles Détaillées - SFD v2.1.0

Edition consolidée - Backend V2, Frontend V2, Legacy V1, architecture trois clients web

Métadonnée	Valeur
Projet	L'épopée des silences
Ancien nom	RPG_ESI07
Document	SFD v2.1.0 - Spécifications fonctionnelles détaillées consolidées
Version cible	app-2.0.0 / alpha roadmap
Client principal	Web - Vue 3 / TypeScript
Décision frontend v2	apps/admin-portal + apps/player-portal + apps/game-client
Legacy	Unity v1 et portail web v1 conservés en legacy
Statut	Mise à jour des SFD avec specs de transition backend et frontend

Résumé exécutif. Cette version consolide les SFD existantes avec les décisions de transition v1 vers v2. Elle confirme le Game Engine comme cœur runtime serveur-autoritaire, intègre le découpage backend en services périphériques, et précise la nouvelle architecture frontend v2 en trois applications web séparées : admin-portal, player-portal et game-client. Le portail web v1 et Unity deviennent des références legacy, non des cibles principales de la roadmap v2.

Sommaire

- 1. Objectif de la mise à jour
- 2. Synthèse des décisions consolidées
- 3. Vision produit et expérience frontend v2
- 4. Architecture cible consolidée
- 5. Découpage frontend v2 en trois applications
- 6. Mapping du frontend actuel vers legacy et nouvelles apps
- 7. Structure cible du repository
- 8. Wireframes et écrans cible mis à jour
- 9. Règles serveur-autoritaire côté frontend
- 10. Intégration des specs backend V1 vers V2
- 11. Roadmap de réalisation
- 12. Actions concrètes à mettre en place
- 13. Critères d'acceptation
- 14. Risques, arbitrages et points ouverts

1. Objectif de la mise à jour

But. Cette mise à jour complète les SFD v2.0.0 sans les contredire. Elle précise la cible frontend v2, formalise le passage du portail web v1 en legacy et intègre les décisions de transition backend déjà actées : Game Engine central, Catalog, Player, Identity, Audit/GDPR, Leaderboard et Event Sourcing ciblé sur les runs.

Inclus dans cette mise à jour	Non inclus / futur document
Architecture frontend v2 en trois apps	Direction artistique finale
Mapping frontend v1 vers legacy/web-v1	Maquettes pixel perfect
Structure cible apps/packages/services/legacy	Implémentation détaillée de tous les écrans
Règles de partage packages/web-shared	Design system complet
Intégration spec backend V1 -> V2	Gameplay combat complet
Roadmap frontend et backend consolidée	Contenu narratif final

2. Synthèse des décisions consolidées

Décision	Justification	Conséquence
Trois applications frontend v2	Les usages joueur, jeu actif et administration sont différents	apps/admin-portal, apps/player-portal, apps/game-client
Le frontend actuel devient legacy web-v1	Il est fortement lié au portail v1 et aux endpoints legacy	Déplacement vers legacy/web-v1, conservation comme référence
Unity reste legacy	La v2 est full web	legacy/unity reste hors cible principale
Game Engine central	Run, events, combat, rewards, laws et narration sont fortement couplés runtime	Pas de microservice par table
Services périphériques isolés	Catalog, Player, Identity, Audit/GDPR et Leaderboard ont des responsabilités durables distinctes	Communication via ports, HTTP/adapters puis events
web-shared minimal	Eviter la duplication entre les trois apps	Partager auth, api, types publics, composants UI génériques, erreurs

3. Vision produit et expérience frontend v2

La v2 n'est pas un simple portail web. Elle devient l'application web principale du jeu. Elle doit couvrir à la fois la mémoire/progression du joueur, l'administration du contenu et l'expérience de jeu en run active.

Espace	Question utilisateur	Nature
Player Portal	Qu'ai-je vécu, débloqué, affronté, écrit dans mon Tome ?	Consultation, historique, progression, collection
Game Client	Que dois-je choisir maintenant dans ma run ?	Interaction immédiate, run active, combat, rewards
Admin Portal	Quel contenu versionné est disponible et publiable ?	Administration, catalog, modération, saisons

Principes UX transverses

- Gameplay principalement souris, lisible et rapide.

- Le client de jeu doit éviter une sensation de dashboard administratif.
- Le portail joueur doit valoriser les runs passées, les seeds rejouables, les ennemis rencontrés, les objets obtenus et le Tome.
- Le client de jeu doit afficher clairement les intentions possibles, mais ne jamais calculer les résultats serveur.
- Le mode alpha peut conserver un panneau JSON/debug pour accélérer les tests manuels.

4. Architecture cible consolidée

SFD v2.1 - Architecture cible consolidee

Trois clients web separes, un Game Engine serveur-autoritaire, services peripheriques isolés.

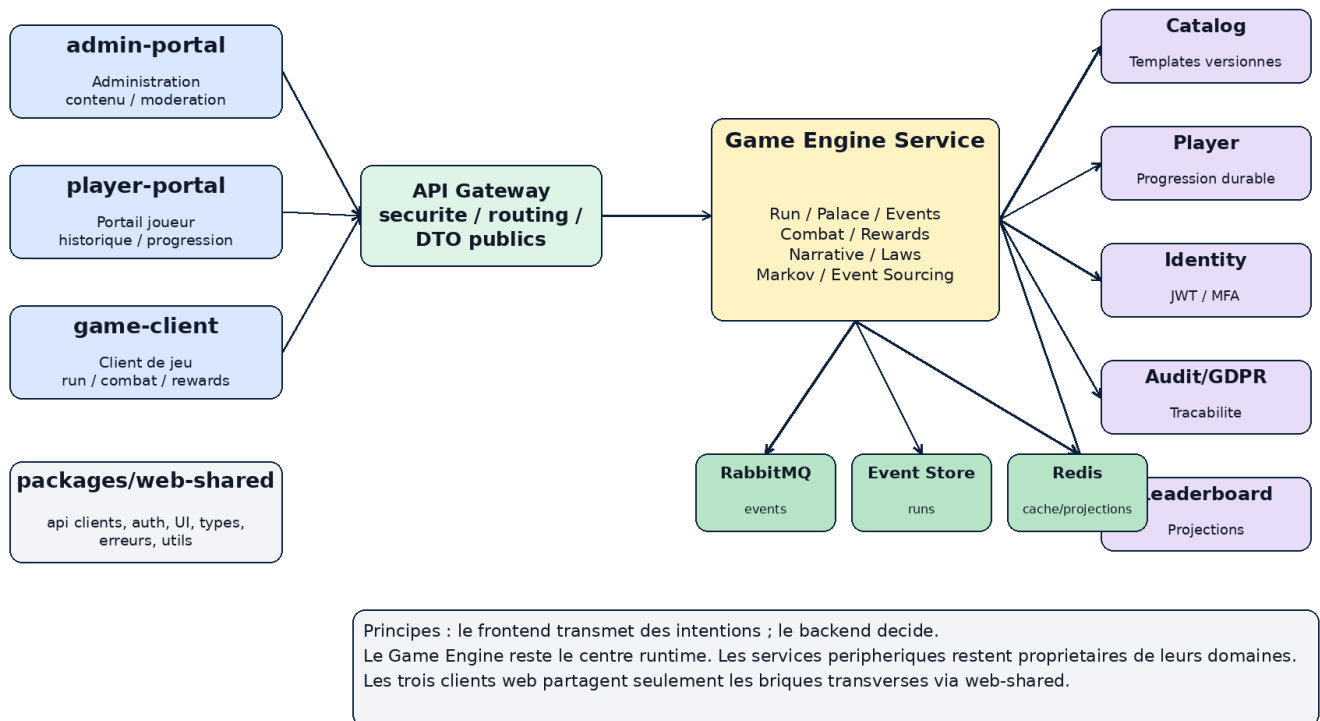


Figure 1 - Architecture cible consolidée : trois clients web, Gateway/API, Game Engine central et services périphériques.

5. Découpage frontend v2 en trois applications

Application	Responsabilité	Routes typiques	Dépendances
apps/player-portal	Dashboard joueur, runs, seeds, collections, Tome, compagnons, stats, leaderboard	/dashboard, /runs, /runs/{id}, /seeds, /tome, /items, /bestiary	web-shared, services Player/Game Engine/Leaderboard via API
apps/game-client	Run active, room map, node choice, event panel, combat, rewards, fragments narratifs	/play, /play/run/{runId}, /play/combat/{combatId}, /play/reward	web-shared, Game Engine API
apps/admin-portal	Administration Catalog, laws, event templates, fragments, saisons, utilisateurs et audit	/admin/catalog, /admin/laws, /admin/events, /admin/seasons	web-shared, Catalog/Admin APIs

Pourquoi ne pas garder une seule app ?

- Les cycles d'évolution diffèrent : le game-client évolue avec la boucle gameplay, le player-portal avec les projections, l'admin avec le contenu.
- Les permissions diffèrent : joueur, admin, modération, lecture publique leaderboard.
- Les layouts diffèrent : dashboard, interface immersive, back-office.
- La séparation réduit le risque de mélange entre UI de jeu et outils d'administration.

6. Mapping du frontend actuel vers legacy et nouvelles apps

Le zip frontend ajouté aux sources correspond au portail Vue v1 : API auth/bestiary/companion/enemies/gameSave/inventory/rgpd/skills/users, vues player/admin/auth, DownloadView, stores auth et tests associés. Il ne doit pas être supprimé : il sert de référence et de banque de composants/tests, mais il ne doit plus recevoir le client jeu v2.

Élément actuel	Destination cible	Décision
src/views/auth/*	packages/web-shared/auth ou reprise progressive dans chaque app	Réutiliser les patterns, ne pas copier aveuglément
src/stores/auth.ts	packages/web-shared/auth	Extraire un auth client commun si stable
src/api/*.ts	packages/web-shared/api ou réécriture par app	Classifier par API cible : legacy, game-engine, catalog, player
src/views/player/DashboardView.vue	apps/player-portal	Migrer conceptuellement, adapter aux DTO v2
src/views/player/GameSavesView.vue	legacy/web-v1	Ne pas migrer tel quel : GameSave remplacé par Event Store/projections
src/views/player/BestiaryView.vue	apps/player-portal/features/collections/enemies	Réutiliser l'idée, adapter au service Player/Catalog
src/views/player/InventoryView.vue	apps/player-portal/features/collections/items	Distinguer objets temporaires de run et permanents
src/views/player/LeaderboardView.vue	apps/player-portal/features/leaderboard	Adapter aux projections v2
src/views/admin/*	apps/admin-portal	Migrer vers administration Catalog v2
src/views/DownloadView.vue	legacy/web-v1	Lié à Unity/download v1
src/tests/*	legacy/web-v1 tests + inspiration pour nouveaux tests	Conserver, puis recréer par app v2

7. Structure cible du repository

Structure cible du repository

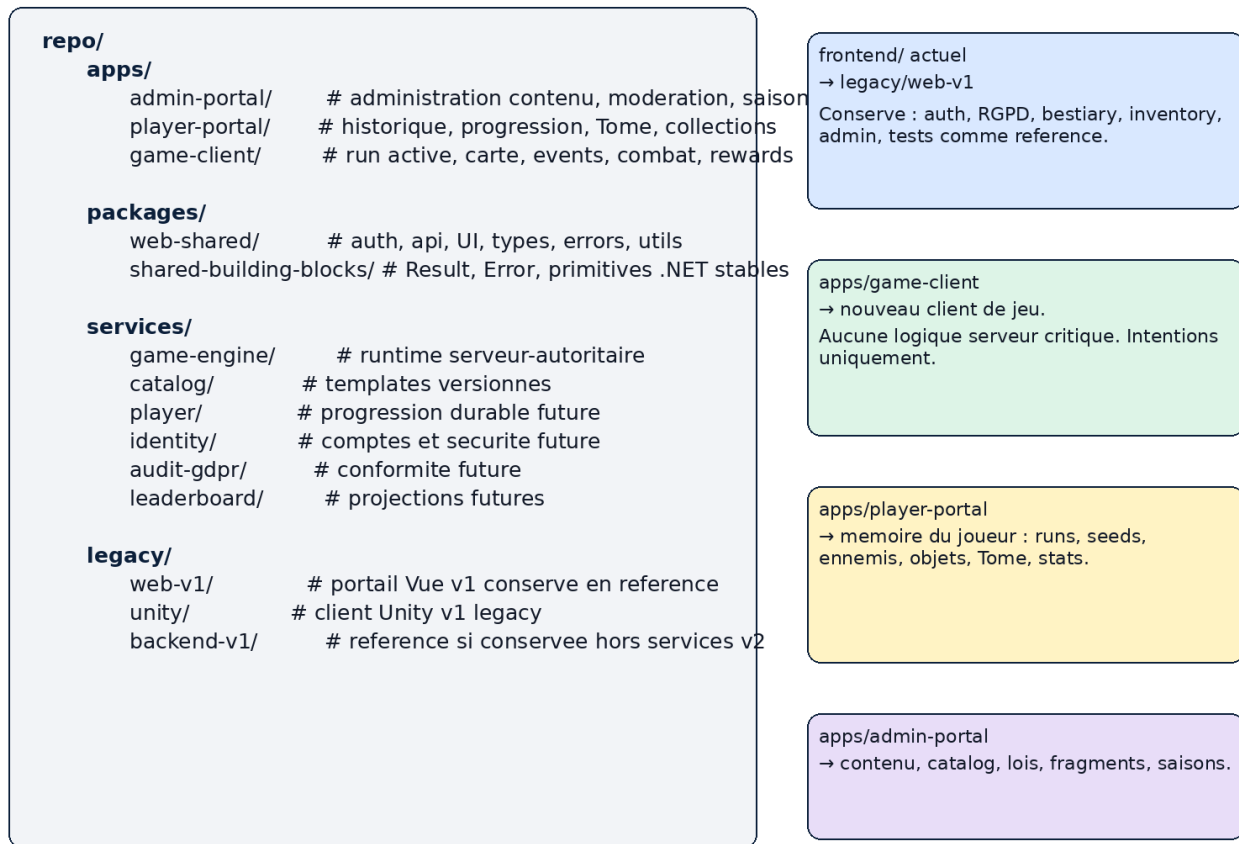


Figure 2 - Structure cible du repository après isolation legacy et préparation des apps v2.

Règles de partage

Emplacement	Autorisé	Interdit
packages/web-shared	auth client générique, API base client, erreurs, composants UI neutres, types publics partagés	logique métier de run, Markov, combat, règles serveur
apps/game-client	UI run, carte, node events, combat/reward panels	calcul dégâts, génération, loot, validation de progression
apps/player-portal	historique, stats, projections, collections	résolution de run active côté client
apps/admin-portal	gestion contenu, versioning templates, publication	accès direct aux données runtime Game Engine
legacy/web-v1	référence historique	nouvelles features v2

8. Wireframes et écrans cible mis à jour

8.1 Player Portal

Wireframe - Player Portal

Espace consultation : progression, historique, collections, Tome et seeds.

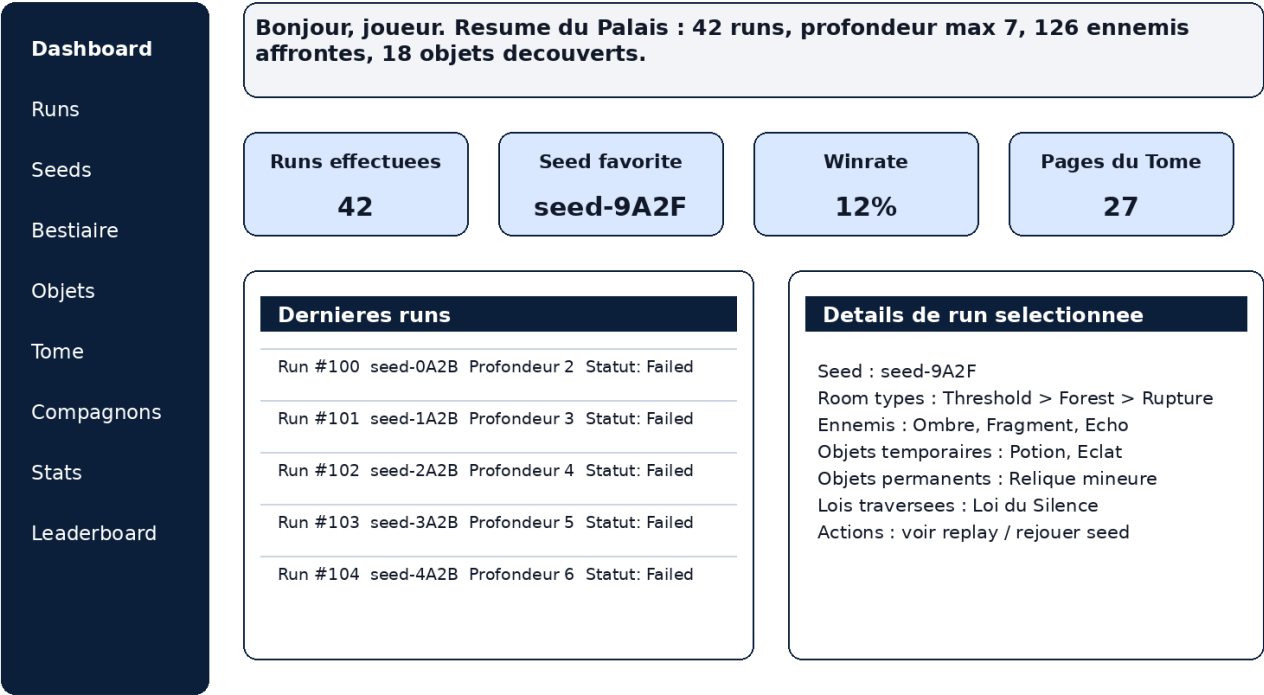


Figure 3 - Wireframe Player Portal : dashboard, historique, seeds et détail de run.

Écrans cibles Player Portal

Route	Fonction	Composants clés
/dashboard	Résumé joueur	KPIs, dernière run, raccourcis
/runs	Liste des runs	Filtres, statut, profondeur, seed, score
/runs/{runId}	Détail de run	Seed, rooms, nodes, ennemis, objets, lois, événements
/seeds	Seeds jouées et rejouables	Favoris, partage, replay
/bestiary	Ennemis rencontrés	Templates rencontrés, stats, lore
/items	Objets récupérés	Temporaire/permanent, rareté, source
/tome	Tome du joueur	Pages, fragments, historique narratif
/companions	Compagnons	Neige, roster, progression
/leaderboard	Classement	Saison, score, seed, run status

8.2 Game Client

Wireframe - Game Client

Espace jeu : run active, carte de room, events, lois, equipe, rewards.

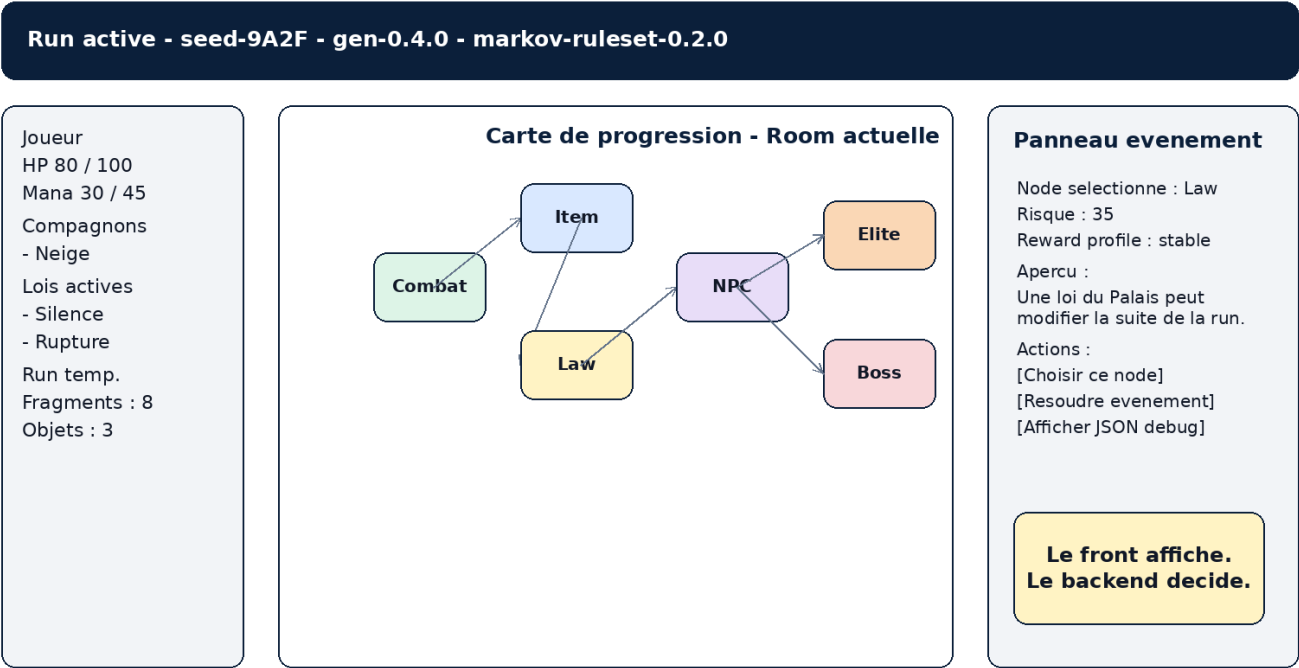


Figure 4 - Wireframe Game Client : run active, carte, panneau événement et état temporaire.

Route	Fonction	Composants clés
/play	Démarrer ou reprendre une run	StartRun, ContinueRun, seed input plus tard
/play/run/{runId}	Écran principal de run	RunHeader, RoomMap, NodeCard, LawsPanel, TeamPanel
/play/run/{runId}/event	Résolution événement	EventPanel, choices, narrative fragment, debug DTO
/play/combat/{combatId}	Combat	CombatBoard, CombatActions, TurnLog
/play/reward	Récompense	RewardOffer, RewardChoice
/play/summary	Résumé	Room summary, run summary, next actions

8.3 Admin Portal

Wireframe - Admin Portal

Espace administration : Catalog, lois, events, fragments, saisons.

Enemies

Items

Skills

Events

Palace Laws

Narrative

Seasons

Users

Audit

Administration du contenu versionne - chaque changement produit une version et reste testable.

Liste des Lois du Palais

Loi du Silence v1

Loi de la Rupture v1

Loi du Reflet v1

Loi du Seuil v1

Edition versionnee

Key : law-silence

Version : 1.0.0

Domaines : generation, events, narrative

Priorite : 50

Statut : active

Actions :

[Creer nouvelle version]

[Desactiver]

[Tester sur seed]

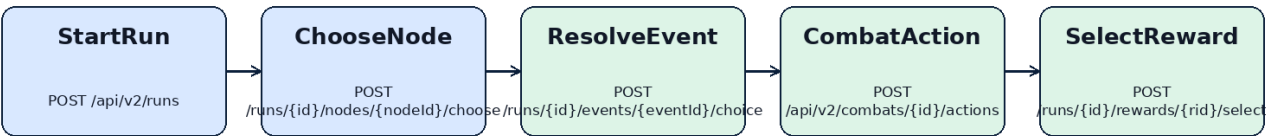
[Publier]

Figure 5 - Wireframe Admin Portal : administration de contenu versionné.

Route	Fonction	Composants clés
/admin/enemies	Templates ennemis	CRUD versionné, scaling, types
/admin/items	Templates objets	Rareté, durée, effets
/admin/skills	Compétences	Coûts, types, effets
/admin/events	Event templates	Conditions, outcomes, tags
/admin/laws	Lois du Palais	Domaines impactés, version, activation
/admin/narrative	Fragments narratifs	Tags, speaker, triggers
/admin/seasons	Saisons leaderboard	Dates, règles, récompenses

9. Règles serveur-autoritaire côté frontend

Flux serveur-autoritaire du game-client



Contraintes :

- le client ne calcule pas la generation, les degats, le loot, la progression, le score ou le leaderboard ;
- le client envoie uniquement une intention ;
- le backend valide l'etat courant, applique les regles, ecrit les evenements et renvoie le nouvel etat ;
- le panneau debug peut afficher les DTO en alpha pour acclereler les tests manuels.

Figure 6 - Flux des intentions frontend vers décisions backend.

Intention frontend	Validation backend	Résultat backend
StartRun	Utilisateur authentifié, pas de run incompatible	RunStarted, seed, room initiale
ChooseNode	Node accessible, run active, état valide	NodeSelected
ResolveEventChoice	Événement courant, choix disponible, contraintes respectées	EventResolved / CombatStarted / RewardOffered
ChooseCombatAction	Tour valide, compétence disponible, cible valide	CombatActionResolved / CombatEnded
SelectReward	Reward proposée et non consommée	RewardSelected, effets appliqués
AbandonRun	Run active	RunAbandoned

10. Intégration des specs backend V1 vers V2

Les décisions de transition backend sont intégrées à ces SFD. Le backend v1 reste une base de connaissance à mapper, pas un brouillon à supprimer. La v2 conserve un Game Engine central pour le runtime gameplay et isole les domaines périphériques.

Service cible	Responsabilité principale	Modèles clés
Game Engine	Runtime serveur-autoritaire	Run, Room, Node, CombatInstance, RewardOffer, ActivePalaceLaws
Catalog	Contenu versionné	EnemyTemplate, SkillTemplate, ItemTemplate, NpcTemplate,

		PalaceLawDefinition, EventTemplate
Player	Progression durable	PlayerProfile, Inventory, Skills, Bestiary, Companions, PlayerCombatStatistics
Identity	Comptes et sécurité	User, JWT, MFA, Argon2id, roles, lockout
Audit/GDPR	Traçabilité et conformité	AuditLog, consentements, export, anonymisation
Leaderboard	Classement et saisons	Run score, seed, season, projections

Mapping V1 -> V2 à conserver

Legacy	Cible v2	Décision
User/Auth/MFA	Identity	Migrer sans perdre les acquis sécurité
Enemy	Catalog + Game Engine	Template catalogue + snapshot runtime
Item	Catalog + Player + Game Engine	Template, possession durable, reward runtime
Skill	Catalog + Player + Game Engine	Template, unlock, snapshot combat
Npc/NpcInteraction	Catalog + Game Engine + Player	Template, event resolver, historique joueur
PlayerProfile	Player	Adapter en snapshot run/combat
CombatStats	Player	Renommer PlayerCombatStatistics
GameSave	Event Store + projections	Ne pas migrer tel quel

11. Roadmap de réalisation

11.1 Roadmap frontend

Jalon	Objectif	Commit recommandé
web-alpha-0.0.1	Isoler legacy web-v1 et préparer workspace apps/packages	chore(frontend): isolate legacy web client and prepare v2 apps
web-alpha-0.0.2	Initialiser packages/web-shared	feat(web-shared): add shared frontend foundations
web-alpha-0.0.3	Créer player-portal skeleton	feat(player-portal): initialize player portal shell
web-alpha-0.0.4	Créer game-client playground	feat(game-client): add v2 run playground
web-alpha-0.0.5	Créer admin-portal skeleton	feat(admin-portal): initialize admin portal shell
web-alpha-0.1.0	Afficher première boucle backend jouable	feat(game-client): display first playable backend loop

11.2 Roadmap backend intégrée

Jalon	Objectif	Commit recommandé
alpha-0.0.7	ResolveNodeEventCommand	feat(game-engine): expose node event resolution use case

alpha-0.0.8	Combat runtime domain	feat(game-engine): introduce combat runtime domain
alpha-0.0.9	Start combat from resolved event	feat(game-engine): start combat from resolved event content
alpha-0.0.10	Combat action flow	feat(game-engine): expose combat action flow
alpha-0.0.11	Reward offer flow	feat(game-engine): introduce reward offer flow
alpha-0.1.0	First backend playable slice	tag alpha-0.1.0

12. Actions concrètes à mettre en place

PR 1 - Isolation legacy et workspace

1. Créer les dossiers apps/, packages/ et legacy/ si absents.
2. Déplacer le dossier frontend actuel vers legacy/web-v1.
3. Conserver legacy/web-v1/package.json, vite.config.ts, tests, src et assets.
4. Déplacer ou vérifier Unity dans legacy/unity.
5. Créer apps/admin-portal, apps/player-portal, apps/game-client avec README.md minimal.
6. Créer packages/web-shared avec README.md minimal.
7. Mettre à jour .gitignore pour ne pas ignorer apps/ ou packages/ métiers.
8. Documenter le déplacement dans docs/v2/follow-up.

```
mkdir apps packages legacy
mkdir apps/admin-portal apps/player-portal apps/game-client packages/web-shared
git mv frontend legacy/web-v1
```

PR 2 - web-shared minimal

- Ajouter un client HTTP générique configurable par variable d'environnement.
- Ajouter types ApiError, ApiResult si côté TS.
- Ajouter auth token storage uniquement si partagé proprement.
- Ajouter composants UI neutres uniquement si stables.

```
packages/web-shared/src/api/httpClient.ts
packages/web-shared/src/auth/authTokenStore.ts
packages/web-shared/src/errors/apiError.ts
packages/web-shared/src/types/index.ts
```

PR 3 - game-client playground

- Créer une route /playground ou /play/run pour démarrer une run.
- Brancher POST /api/v2/runs et GET /api/v2/runs/{runId}.
- Afficher la room courante, les nodes et les états.
- Ajouter les actions ChooseNode, ResolveSelectedNode et GenerateNextNodes si endpoints disponibles.
- Afficher un panneau debug JSON en alpha.

13. Critères d'acceptation

- Le frontend v1 est isolé dans legacy/web-v1 sans perte d'historique Git lorsque possible.
- Les trois apps v2 existent dans apps/ avec responsabilités documentées.
- Aucune nouvelle feature v2 n'est ajoutée dans legacy/web-v1.
- packages/web-shared ne contient pas de logique métier gameplay.
- Le game-client n'effectue aucun calcul critique côté client.
- La documentation explique quoi migrer, quoi réécrire et quoi conserver uniquement comme référence.

- Les tests existants du legacy restent conservés ; les nouvelles apps introduisent leurs propres tests progressivement.

14. Risques, arbitrages et points ouverts

Risque	Impact	Réponse
Trop de duplication entre trois apps	Maintenance lourde	Créer web-shared minimal, pas de duplication auth/api
web-shared devient fourre-tout	Couplage transverse	Limiter aux briques stables et neutres
game-client simule le backend	Violation serveur-autoritaire	Afficher uniquement l'état serveur et envoyer des intentions
legacy web-v1 continue à évoluer	Confusion v1/v2	Interdire nouvelles features v2 dans legacy
Admin trop tôt	Charge excessive	Skeleton uniquement avant Catalog avancé
Player Portal dépend de projections non prêtes	Écrans vides	Commencer par placeholders et DTO alpha documentés

Points ouverts

- Choix final du workspace : npm workspaces, pnpm workspace ou maintien npm simple.
- Design system : Vuetify conservé, remplacé ou complété par composants maison.
- Déploiement : trois frontends séparés ou déploiement unique avec sous-chemins.
- Auth : Identity legacy temporaire ou extraction Identity avant player-portal complet.
- Niveau de réutilisation exact des vues v1 : copier seulement après audit composant par composant.

Conclusion

Décision consolidée. La v2 devient une application web de jeu structurée autour de trois clients : admin-portal, player-portal et game-client. Cette séparation clarifie les responsabilités, évite de mélanger portail joueur, gameplay actif et administration, et prépare une évolution propre vers la boucle jouable alpha-0.1. Le frontend v1 est conservé en legacy/web-v1 comme référence et source d'éléments réutilisables, tandis que le Game Engine reste le centre runtime serveur-autoritaire.